Server & Application Security Hardening Guide

A general-purpose guide for VPS owners, indie developers, and small teams

Edition 2026 · Covers Ubuntu/Debian + Nginx + Docker + Python/Node apps

Whether you run a personal project or a small SaaS, your VPS is a target. Automated scanners probe every public IP within minutes of provisioning. This guide distils the most impactful security practices for a typical Linux VPS running Nginx, Docker-based apps, and Python or Node services — organised by domain, with clear bash commands so you can act immediately.

■■ A misconfigured server can be compromised within hours. The fixes in this guide are free, battle-tested, and require no paid tools. Most can be applied in under two hours.

What Claude Code Can Do Automatically vs. Manually

Many hardening steps are pure configuration changes — Claude Code can SSH into your server and apply them without you typing a single command. Others require your input, credentials, or decisions about your specific setup.

Nginx security headers (CSP, HSTS, …)	**AUTO**
TLS 1.0 / 1.1 disabled in Nginx	**AUTO**
Rate limiting zones in nginx.conf	**AUTO**
Fail2Ban installation & SSH jail config	**AUTO**
robots.txt AI-bot blocklist	**AUTO**
Removing/stopping crash-looping services	**AUTO**
Docker port binding to localhost	**AUTO**
SSH key generation (key pair created for you)	**AUTO**
SSH hardening (PermitRootLogin, Port, …)	**MANUAL — requires you to add the public key first**
UFW Cloudflare IP allowlisting	**MANUAL — need Cloudflare account**
Systemd service non-root user	**MANUAL — depends on your app path**
Restic encrypted backups to Backblaze B2	**MANUAL — needs B2 account + bucket**
Log shipping via RSyslog → Better Stack	**MANUAL — needs Better Stack source token**
Cloudflare WAF & proxy setup	**MANUAL — DNS migration required**
SMTP / API secret rotation	**MANUAL — must be done in the external provider**

Severity Scale

Severity	Check Item	Action if Missing
CRITICAL	Actively exploitable right now	Fix immediately — hours, not days
HIGH	Likely to be exploited soon	Fix within 48 hours
MEDIUM	Meaningful risk gap	Fix in next maintenance window
LOW	Best-practice gap	Fix when convenient
INFO	Informational	No action required

1 SSH & Server Access Hardening

SSH is the front door to your server. Every VPS on a public IP receives thousands of login attempts per day from automated bots. The following controls reduce your attack surface to near zero.

1.1 Create a dedicated admin user (non-root)

MANUAL — Run once on a fresh server before locking out root.

```
# Create admin user and add to sudo group
sudo adduser admin
sudo usermod -aG sudo admin

# Verify
sudo -u admin sudo whoami    # should print: root
```

1.2 Generate an Ed25519 SSH key pair (on your local machine)

MANUAL — Run on YOUR Mac/PC, not the server.

```
# On your local machine
ssh-keygen -t ed25519 -C "your@email.com" -f ~/.ssh/server_key

# Copy your public key to the server (while password auth still works)
ssh-copy-id -i ~/.ssh/server_key.pub admin@YOUR_SERVER_IP

# Test key login before disabling password auth
ssh -i ~/.ssh/server_key admin@YOUR_SERVER_IP
```

Add a local SSH config entry so you never need to type the key path:

```
# ~/.ssh/config  (on your local machine)
Host myserver
    HostName YOUR_SERVER_IP
    User admin
    Port YOUR_SSH_PORT
    IdentityFile ~/.ssh/server_key
```

1.3 Harden /etc/ssh/sshd_config

MANUAL — Only do this AFTER you have confirmed key login works.

```
sudo nano /etc/ssh/sshd_config

# Set or uncomment these lines:
Port 56921                  # pick any port 49152-65535
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
LoginGraceTime 30
X11Forwarding no
AllowAgentForwarding no

# Restart SSH (keep current session open until you verify new session works)
sudo systemctl restart sshd

# In a NEW terminal, test the new port before closing the old session
ssh -p 56921 admin@YOUR_SERVER_IP
```

1.4 Install & configure Fail2Ban

AUTO — Claude Code can install and configure this for you.

```
sudo apt install fail2ban -y

sudo tee /etc/fail2ban/jail.local > /dev/null << 'EOF'
[DEFAULT]
bantime  = 7200
findtime = 600
maxretry = 3

[sshd]
enabled  = true
port     = YOUR_SSH_PORT
logpath  = /var/log/auth.log
EOF

sudo systemctl enable --now fail2ban
sudo fail2ban-client status sshd
```

1.5 Enable automatic security updates

AUTO — Claude Code can enable this for you.

```
sudo apt install unattended-upgrades -y
sudo dpkg-reconfigure --priority=low unattended-upgrades
# Choose "Yes" to enable automatic updates

# Verify
sudo systemctl status unattended-upgrades
```

2 Firewall & Network Security

A firewall limits what the outside world can reach on your server. Combined with Cloudflare proxying, you can lock down your origin so that only Cloudflare IPs (and your SSH port) are allowed through.

2.1 UFW baseline setup

AUTO — Claude Code can configure UFW for you.

```
sudo apt install ufw -y

# Block everything by default, allow outbound
sudo ufw default deny incoming
sudo ufw default allow outgoing

# Allow your custom SSH port (replace 56921 with yours)
sudo ufw allow 56921/tcp

# Allow web traffic
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp

sudo ufw enable
sudo ufw status verbose
```

2.2 Cloudflare IP allowlisting (lock origin to Cloudflare only)

> **MANUAL — Only do this after you have moved your domain to Cloudflare and confirmed the proxy works.**

Replace the open port 80/443 rules with Cloudflare IP ranges:

```
# Remove open web rules
sudo ufw delete allow 80/tcp
sudo ufw delete allow 443/tcp
# (if you had a combined rule)
sudo ufw delete allow 80,443/tcp

# Cloudflare IPv4 ranges (check https://cloudflare.com/ips for latest)
for ip in \
  173.245.48.0/20 103.21.244.0/22 103.22.200.0/22 \
  103.31.4.0/22 141.101.64.0/18 108.162.192.0/18 \
  190.93.240.0/20 188.114.96.0/20 197.234.240.0/22 \
  198.41.128.0/17 162.158.0.0/15 104.16.0.0/13 \
  104.24.0.0/14 172.64.0.0/13 131.0.72.0/22; do
  sudo ufw allow from $ip to any port 80,443
done

# Cloudflare IPv6 ranges
for ip in \
  2400:cb00::/32 2606:4700::/32 2803:f800::/32 \
  2405:b500::/32 2405:8100::/32 2a06:98c0::/29 \
  2c0f:f248::/32; do
  sudo ufw allow from $ip to any port 80,443
done

sudo ufw status numbered
```

2.3 Bind Docker services to localhost (not 0.0.0.0)

CRITICAL — Docker bypasses UFW by writing directly to iptables. A container bound to 0.0.0.0:PORT is publicly reachable even if UFW blocks that port.

```
# In your docker-compose.yml, change:
#   ports:
#     - "5678:5678"         ← exposed on all interfaces
# To:
#   ports:
#     - "127.0.0.1:5678:5678"   ← localhost only

# After editing, restart the container
docker compose down && docker compose up -d

# Verify: this should show 127.0.0.1:5678, not 0.0.0.0:5678
sudo ss -tlnp | grep 5678
```

3 Nginx Web Server & TLS Hardening

Nginx sits in front of your apps and is the first thing browsers interact with. Proper TLS configuration, security headers, and rate limiting prevent a wide range of attacks — from protocol downgrade attacks to credential stuffing.

3.1 Disable TLS 1.0 and 1.1 in /etc/nginx/nginx.conf

AUTO — Claude Code can apply this change.

```
# In the http {} block of /etc/nginx/nginx.conf
ssl_protocols TLSv1.2 TLSv1.3;
ssl_prefer_server_ciphers on;
ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384;
server_tokens off;   # hide Nginx version from headers
```

3.2 Rate limiting zones

AUTO — Claude Code can apply this change.

```
# Add to the http {} block in /etc/nginx/nginx.conf
limit_req_zone $binary_remote_addr zone=api:10m     rate=30r/m;
limit_req_zone $binary_remote_addr zone=general:10m rate=120r/m;
limit_req_zone $binary_remote_addr zone=login:10m   rate=5r/m;
```

Then apply zones in your site config:

```
location /api/ {
    limit_req zone=api burst=10 nodelay;
    limit_req_status 429;
    proxy_pass http://127.0.0.1:8000;
}

location /login {
    limit_req zone=login burst=3 nodelay;
    limit_req_status 429;
    proxy_pass http://127.0.0.1:8000;
}

location / {
    limit_req zone=general burst=30 nodelay;
    limit_req_status 429;
    proxy_pass http://127.0.0.1:8000;
}
```

3.3 Security headers (per virtual host)

AUTO — Claude Code can add all security headers to your Nginx site config.

```
# Add inside your server {} block (HTTPS listener)
add_header Strict-Transport-Security "max-age=63072000; includeSubDomains; preload" always;
add_header X-Frame-Options "DENY" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Permissions-Policy "camera=(), microphone=(), geolocation=(), payment=()" always;
add_header Cross-Origin-Opener-Policy "same-origin" always;
add_header Cross-Origin-Resource-Policy "same-origin" always;
add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; font-src 'self'; object-src 'none'; base-uri 'self'; form-action 'self'; frame-ancestors 'none'; upgrade-insecure-requests;" always;

# After editing any Nginx config:
sudo nginx -t && sudo systemctl reload nginx
```

3.4 HTTPS redirect (HTTP → HTTPS)

AUTO — Claude Code can configure this.

```
server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com;
    return 301 https://$host$request_uri;
}
```

3.5 Block AI scrapers and aggressive bots via robots.txt

AUTO — Claude Code can write this file for you.

```
# /var/www/yoursite/robots.txt

User-agent: GPTBot

Disallow: /


User-agent: ChatGPT-User

Disallow: /


User-agent: CCBot

Disallow: /


User-agent: anthropic-ai

Disallow: /


User-agent: Claude-Web

Disallow: /


User-agent: Bytespider

Disallow: /


User-agent: AhrefsBot

Disallow: /


User-agent: SemrushBot

Disallow: /


User-agent: MJ12bot

Disallow: /


User-agent: *

Allow: /
```

4 Application Security

Your application code and its runtime configuration are often the weakest link. The following controls apply regardless of framework (Python/Flask/Django, Node/Express, etc.).

4.1 Run your app as a non-root user (systemd service)

MANUAL — Depends on your app's file path and service name.

```
# Create a dedicated app user (no login shell, no home)
sudo useradd --system --no-create-home --shell /usr/sbin/nologin appuser

# Give the user ownership of the app directory
sudo chown -R appuser:appuser /opt/yourapp

# Edit the systemd service file
sudo nano /etc/systemd/system/yourapp.service

# Make sure these lines are set:
# [Service]
# User=appuser
# Group=appuser

# Reload and restart
sudo systemctl daemon-reload
sudo systemctl restart yourapp
sudo systemctl status yourapp

# Verify the process no longer runs as root
ps aux | grep yourapp
```

4.2 Never hardcode secrets — use environment variables

AUTO — Claude Code can scan your source code for hardcoded secrets.

```
# ■ Never do this in source code:
DB_PASSWORD = "mysecretpassword"

# ■ Do this instead — load from environment:
import os
DB_PASSWORD = os.environ["DB_PASSWORD"]

# Store secrets in a .env file that is NOT committed to git:
# .env
DB_PASSWORD=mysecretpassword

# Ensure .env is in .gitignore:
echo ".env" >> .gitignore

# Scan for accidentally committed secrets:
pip install trufflehog
trufflehog git file://. --no-update
```

4.3 Audit Python dependencies for CVEs

AUTO — Claude Code can run this audit for you.

```
pip install pip-audit
pip-audit -r requirements.txt

# Or with poetry:
pip-audit

# Add to CI/CD (GitHub Actions example):
# - name: Audit dependencies
#   run: pip-audit -r requirements.txt
```

4.4 Secure session cookies

```
# Flask example
app.config["SESSION_COOKIE_HTTPONLY"] = True
app.config["SESSION_COOKIE_SECURE"]   = True      # HTTPS only
app.config["SESSION_COOKIE_SAMESITE"] = "Lax"

# Django (settings.py)
SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_SECURE   = True
SESSION_COOKIE_SAMESITE = "Lax"
CSRF_COOKIE_SECURE      = True
```

4.5 Disable DEBUG mode in production

AUTO — Claude Code can scan your config files for DEBUG=True.

```
# Flask — set via environment variable, not hardcoded:
import os
app.debug = os.environ.get("FLASK_DEBUG", "false").lower() == "true"

# Django
DEBUG = os.environ.get("DJANGO_DEBUG", "False") == "True"
ALLOWED_HOSTS = ["yourdomain.com"]    # never use ['*'] in production

# In your .env (production):
FLASK_DEBUG=false
DJANGO_DEBUG=False
```

5 Database Hardening

Databases should never be directly reachable from the internet. Even on a single-server setup, proper binding, authentication, and least-privilege access limits the blast radius of an application-level breach.

5.1 Bind databases to localhost only

MANUAL — Check each database service separately.

```
# PostgreSQL — /etc/postgresql/*/main/postgresql.conf
listen_addresses = 'localhost'

# MySQL/MariaDB — /etc/mysql/mysql.conf.d/mysqld.cnf
bind-address = 127.0.0.1

# Redis — /etc/redis/redis.conf
bind 127.0.0.1 ::1
protected-mode yes

# Verify nothing is exposed on 0.0.0.0:
sudo ss -tlnp | grep -E "5432|3306|6379|27017"

# None of the above should show 0.0.0.0
```

5.2 Use a least-privilege database user for your app

```
# PostgreSQL — create a restricted user
psql -U postgres
CREATE USER appuser WITH PASSWORD 'strong_random_password';
GRANT CONNECT ON DATABASE myapp TO appuser;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO appuser;
-- Do NOT grant SUPERUSER, CREATEROLE, or CREATEDB

# MySQL
CREATE USER 'appuser'@'localhost' IDENTIFIED BY 'strong_random_password';
GRANT SELECT, INSERT, UPDATE, DELETE ON myapp.* TO 'appuser'@'localhost';
```

5.3 Use strong authentication (PostgreSQL)

```
# /etc/postgresql/*/main/pg_hba.conf
# Change 'md5' to 'scram-sha-256' for all local connections
# host  all  all  127.0.0.1/32  scram-sha-256

# Set the default auth method
sudo -u postgres psql -c "ALTER SYSTEM SET password_encryption = 'scram-sha-256';"
sudo systemctl restart postgresql
```

6 Encrypted Off-Site Backups

Ransomware and hardware failure are real risks. The 3-2-1 rule: 3 copies, 2 media types, 1 off-site. Restic is a free, open-source backup tool that deduplicates and encrypts backups before sending them to Backblaze B2 (cheapest S3-compatible storage).

6.1 Install Restic

> **MANUAL — Requires a Backblaze B2 account (free tier available).**

```
sudo apt install restic -y
restic version
```

6.2 Create a Backblaze B2 bucket and application key

1. Sign up at backblaze.com → go to B2 Cloud Storage → Create Bucket
2. Set: Private, Server-Side Encryption ON, Object Lock OFF
3. Go to App Keys → Add a New Application Key → Read & Write on your bucket
4. Copy the Key ID and Application Key — you only see the key once

6.3 Initialise the Restic repository

```
export B2_ACCOUNT_ID="YOUR_KEY_ID"
export B2_ACCOUNT_KEY="YOUR_APP_KEY"
export RESTIC_REPOSITORY="s3:s3.eu-central-003.backblazeb2.com/YOUR_BUCKET_NAME"
export RESTIC_PASSWORD="YOUR_STRONG_REPO_PASSWORD"

# Initialise (only run once)
restic init

# Test backup
restic backup /opt/yourapp /var/www /srv

# Verify snapshots
restic snapshots
```

6.4 Automated daily backup script

```
sudo tee /usr/local/bin/backup.sh > /dev/null << 'SCRIPT'
#!/bin/bash
export B2_ACCOUNT_ID="YOUR_KEY_ID"
export B2_ACCOUNT_KEY="YOUR_APP_KEY"
export RESTIC_REPOSITORY="s3:s3.eu-central-003.backblazeb2.com/YOUR_BUCKET"
export RESTIC_PASSWORD="YOUR_STRONG_REPO_PASSWORD"

/usr/bin/restic backup /opt /var/www /srv /home --exclude="*.log"
/usr/bin/restic forget --prune --keep-daily 7 --keep-weekly 4 --keep-monthly 3
SCRIPT

sudo chmod +x /usr/local/bin/backup.sh

# Schedule daily at 3am
(crontab -l 2>/dev/null; echo "0 3 * * * /usr/local/bin/backup.sh >> /var/log/restic.log 2>&1") |
crontab -

# Test restore (dry run)
restic restore latest --target /tmp/restore-test --dry-run
```

7 Centralised Off-Server Logging

If an attacker compromises your server, the first thing they do is wipe the logs. Shipping logs to an external service in real-time means you retain forensic evidence even after a breach. Better Stack (Logtail) offers a generous free tier.

7.1 Create a Better Stack source

1. Sign up at betterstack.com/logs
2. Sources → Connect source → Select **Rsyslog**
3. Copy the Source Token shown (format: xxxx…xxxx)
4. Note the ingestion hostname shown (e.g. in.logtail.com)

7.2 Configure RSyslog to forward logs (recommended method)

> **This method uses RSyslog's native TLS forwarding — no agent, no extra process. Works on any Ubuntu/Debian server with rsyslog already installed.**

```
# Install rsyslog TLS module
sudo apt install rsyslog-gnutls -y

# Create the forwarding config
sudo tee /etc/rsyslog.d/logtail.conf > /dev/null << 'EOF'
# Forward all logs to Better Stack via TLS
action(
  type="omfwd"
  target="in.logtail.com"
  port="6514"
  protocol="tcp"
  template="RSYSLOG_SyslogProtocol23Format"
  TCP_Framing="octet-counted"
  StreamDriver="gtls"
  StreamDriverMode="1"
  StreamDriverAuthMode="x509/name"
  StreamDriverPermittedPeers="*.logtail.com"
)
EOF

# Set your token in rsyslog's structured data
sudo tee /etc/rsyslog.d/logtail-token.conf > /dev/null << 'EOF'
template(name="BetterStackFormat" type="string"
  string="<%pri%>1 %timestamp:::date-rfc3339% %hostname% %app-name% %procid% %msgid% [YOUR_SOURCE_TOKEN@51137] %msg%\n"
)
action(
  type="omfwd"
  target="in.logtail.com"
  port="6514"
  protocol="tcp"
  template="BetterStackFormat"
  TCP_Framing="octet-counted"
  StreamDriver="gtls"
  StreamDriverMode="1"
  StreamDriverAuthMode="x509/name"
```

```
  StreamDriverPermittedPeers="*.logtail.com"
)
EOF

sudo systemctl restart rsyslog
sudo systemctl status rsyslog

# Verify logs appear in Better Stack Live Tail within ~30 seconds
```

7.3 Set up alerts in Better Stack

Go to Alerts → Create alert with these queries to get notified of critical events:

Root login attempt	message:"Accepted password for root"
New sudo usage	message:"sudo:"
SSH brute-force	message:"Failed password" count > 10 in 1min
Nginx 5xx errors	message:"HTTP/1" status:5[0-9][0-9]

8 Cloudflare WAF & Edge Protection

Cloudflare's free tier provides a WAF, DDoS protection, TLS termination, and caching — all without touching your server. Proxying your domain through Cloudflare hides your origin IP and absorbs malicious traffic before it reaches Nginx.

8.1 Add your domain to Cloudflare

> **MANUAL — Requires DNS migration to Cloudflare nameservers.**

1. Sign up at cloudflare.com → Add site → Enter your domain
2. Cloudflare will scan your existing DNS records
3. Update your domain's nameservers at your registrar to the ones Cloudflare provides
4. Wait for propagation (usually 5–30 minutes)

8.2 Enable proxy (orange cloud) for A records

In Cloudflare DNS → for each A record pointing to your server → click the cloud icon until it turns orange. This routes all traffic through Cloudflare.

8.3 Enable key Cloudflare settings

SSL/TLS mode	Set to Full (Strict) — encrypts both Cloudflare→browser and Cloudflare→origin
Always Use HTTPS	SSL/TLS → Edge Certificates → Always Use HTTPS: ON
Universal SSL	SSL/TLS → Edge Certificates → Universal SSL: ON (free cert)
WebSockets	Network → WebSockets: ON (required for real-time apps like n8n)
Bot Fight Mode	Security → Bots → Bot Fight Mode: ON (free bot mitigation)
Security Level	Security → Settings → Security Level: Medium or High
HSTS (Edge)	SSL/TLS → Edge Certificates → HSTS → Enable, max-age 12 months

8.4 Authenticated Origin Pulls (prevent bypassing Cloudflare)

> **Without this, an attacker who discovers your origin IP can hit it directly, bypassing the WAF.**

```
# In Cloudflare: SSL/TLS → Origin Server → Authenticated Origin Pulls → ON

# On your server, download Cloudflare's CA cert and configure Nginx to require it:
sudo wget -O /etc/nginx/cloudflare-origin-pull-ca.pem \
  https://developers.cloudflare.com/ssl/static/authenticated_origin_pull_ca.pem

# In your Nginx server block (HTTPS):
ssl_client_certificate /etc/nginx/cloudflare-origin-pull-ca.pem;
ssl_verify_client on;

sudo nginx -t && sudo systemctl reload nginx
```

9 Secrets Management & CI/CD Security

Exposed API keys and database passwords in git history are one of the most common causes of production breaches. Adding automatic scanning to your workflow catches these before they are ever committed.

9.1 Pre-commit secrets scanning with Gitleaks

AUTO — Claude Code can install and configure this for you.

```
# Install gitleaks
brew install gitleaks                        # macOS
# or: sudo apt install gitleaks             # Ubuntu

# Scan entire git history for secrets
gitleaks detect --source . --report-format json --report-path leaks.json

# Install as a pre-commit hook
pip install pre-commit
cat > .pre-commit-config.yaml << 'EOF'
repos:
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.18.0
    hooks:
      - id: gitleaks
EOF
pre-commit install
```

9.2 Rotate any exposed secrets immediately

If a secret has EVER been committed to a public or semi-public repo, assume it is compromised. Rotate it even if you deleted it — git history is permanent.

Rotation checklist:

SMTP password	Change in your email provider settings, update .env on server
Database password	ALTER USER in DB, update .env, restart app
API keys	Revoke old key in provider dashboard, generate new, update .env
Webhook secrets	Regenerate in the webhook provider, update your app config
Log source tokens	Delete old source in Better Stack, create new source, update rsyslog.d/

9.3 GitHub Actions security

```
# Store secrets in GitHub repo Settings → Secrets → Actions
# Access in workflow:
env:
  DB_PASSWORD: ${{ secrets.DB_PASSWORD }}

# Never echo secrets in CI logs
# Audit third-party actions — only use pinned SHA refs:
# uses: actions/checkout@b4ffde65f46336ab88eb53be808477a3936bae11  # v4.1.1
```

10 Quick Self-Audit Checklist

Run these commands on your server to get an instant read on your security posture.

```
# ■■ SSH ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
grep -E "^PasswordAuthentication|^PermitRootLogin|^Port|^MaxAuthTries|^LoginGraceTime" \
  /etc/ssh/sshd_config

# ■■ Firewall ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sudo ufw status verbose

# ■■ All listening services ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sudo ss -tlnp

# ■■ Nginx security headers ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
curl -sI https://yourdomain.com | grep -iE "strict-transport|x-frame|content-security|x-content-type"

# ■■ TLS protocol (external) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Visit: https://www.ssllabs.com/ssltest/

# ■■ Fail2Ban ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sudo fail2ban-client status sshd

# ■■ App running as non-root ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ps aux | grep -E "gunicorn|uvicorn|node|python" | grep -v root

# ■■ Docker ports ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
docker ps --format "table {{.Names}}\t{{.Ports}}"
# Danger: any entry showing 0.0.0.0:PORT is exposed on all interfaces

# ■■ Database binding ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sudo ss -tlnp | grep -E "5432|3306|6379"

# ■■ Backups ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
crontab -l | grep backup
restic snapshots    # requires RESTIC_REPOSITORY and RESTIC_PASSWORD set

# ■■ Secrets in source code ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
grep -rE "(SECRET|PASSWORD|API_KEY|TOKEN)\s*=" ./app --include="*.py" --include="*.js"
```

```
# ■■ Dependency CVEs (Python) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
pip-audit -r requirements.txt
```

Useful External Tools

SSL Labs	TLS / certificate grade — ssllabs.com/ssltest
Security Headers	HTTP header grade — securityheaders.com
Mozilla SSL	Nginx TLS config generator — ssl-config.mozilla.org
Shodan	External port scan view — shodan.io
Have I Been Pwned	Check if credentials leaked — haveibeenpwned.com
pip-audit	Python CVE scanner — pip install pip-audit
Gitleaks	Secrets in git history — github.com/gitleaks/gitleaks
OWASP ZAP	Dynamic app scanning — zaproxy.org
DNS Checker	DNS propagation checker — dnschecker.org
CIS Benchmarks	OS hardening baselines — cisecurity.org/cis-benchmarks

How to Rate Your Security After Applying This Guide

Severity	Check Item	Action if Missing
CRITICAL	Any CRITICAL finding outstanding	Not safe for production
HIGH	All CRITICAL fixed, HIGH items remain	Better but still exposed
MEDIUM	All CRITICAL + HIGH fixed	Solid baseline — 85/100
LOW	All above fixed + backups + logging active	Strong posture — 95/100
INFO	All above + WAF + CI scanning + SBOM	Industry-grade — 100/100

■ After applying all steps in this guide your server reaches a 95/100 security posture — stronger than the vast majority of small VPS deployments. Re-audit every 3–6 months or after any major infrastructure change.

Disclaimer & Legal Notice

This guide has been compiled thoroughly and in good faith to reflect current best practices for server and application security hardening. However, the publisher of this guideline takes no responsibility whatsoever for the correctness, completeness, or accuracy of the information provided, nor for any damage, data loss, service disruption, security breach, or any other harm that may result directly or indirectly from the use or misuse of the instructions contained herein.

The user is solely responsible for every action taken on their systems based on this guide. Security configurations are highly environment-specific and what works correctly in one setup may have unintended consequences in another. Always test changes in a staging environment before applying them to production systems.

For production systems in particular, it is strongly recommended to consult a qualified IT security specialist or a certified penetration tester before going live. A professional review ensures that all configurations are appropriate for your specific infrastructure, compliance requirements, and threat model — and helps guarantee total security for your environment.

For production systems, always consult a qualified IT security specialist before going live. Professional advice ensures your configuration is appropriate for your specific environment and threat model.